

# ColorRoom

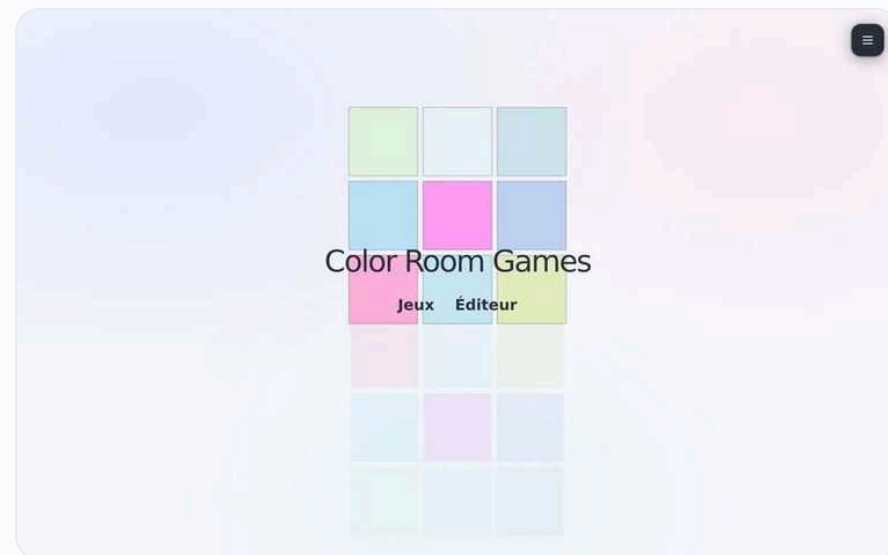


Serious games pédagogiques sur les plaques lumineuses de la ColorRoom

**Téo Trompier** · partie E2

Équipe JavaScript · Lycée Édouard Branly, Lyon

Partenaires : LUMEN – La Cité de la Lumière · ENTPE / LTDS / BPMNP





# Sommaire

---



Contexte et système



Architecture et classes



Réseau et déploiement



Sécurité et comptes



Mesure et chromaticité



Bilan



Cas d'utilisation et équipe



Choix techniques et stack



Interface et données



Jeux, IA et multijoueur



Documentation et qualité



Démonstration

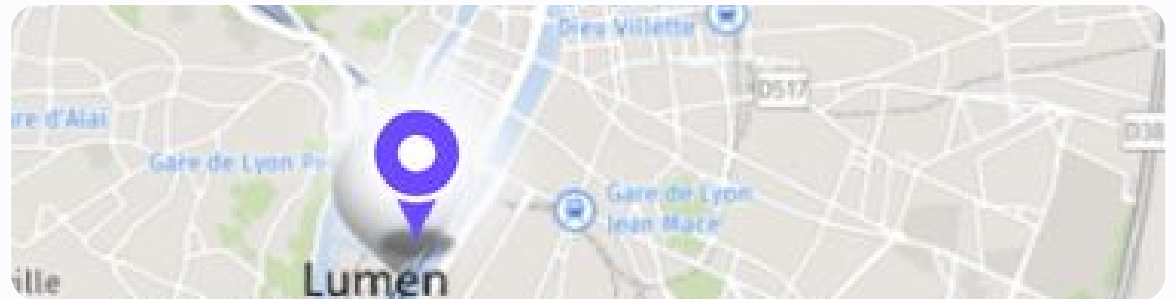


# Contexte et partenaire

- Commanditaire : laboratoire de recherche **ENTPE / LTDS**, équipe **BPMNP**
- ColorRoom hébergée à **LUMEN – La Cité de la Lumière** (Lyon Confluence)
- Équipement scientifique : **2 cellules jumelles** d'analyse des effets colorés
- Éclairages à **spectres précis** et hautes intensités
  - Contacts : M. Labayrade, M. Vella · Professeur : M. Delbosc



 LUMEN · Cité de la Lumière (Lyon Confluence)

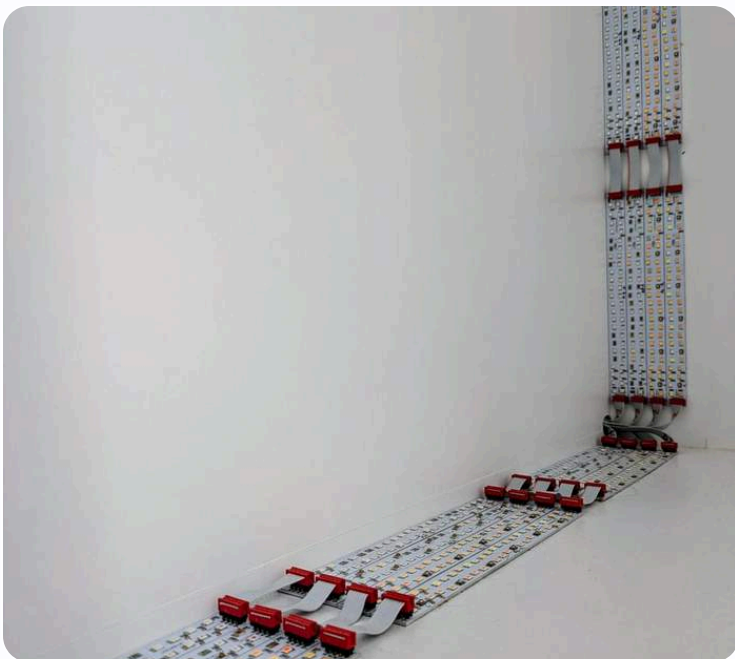


 Implantation : LUMEN & ENTPE, agglomération lyonnaise



UNE INSTALLATION LUMINEUSE UNIQUE

# Le système ColorRoom



⚡ LED en bandes sur les bords d'une plaque · 2360 LED, ~300 W



## 42

plaques (21 par cellule)



## 32

canaux = 24 spectres étroits + 8 blancs



## 2360

LED par plaque (~300 W max)



## RS-485

bus de pilotage des dalles



# Problématique et objectifs

---

Le logiciel de recherche de la ColorRoom est trop complexe pour l'initiation. Comment rendre la salle accessible aux apprenants, de façon ludique et pédagogique ?

## Objectifs

- Une série de **serious games** sur la lumière et la couleur
- Accessible depuis un navigateur, sans installation
- Adaptable à une seule plaque lumineuse

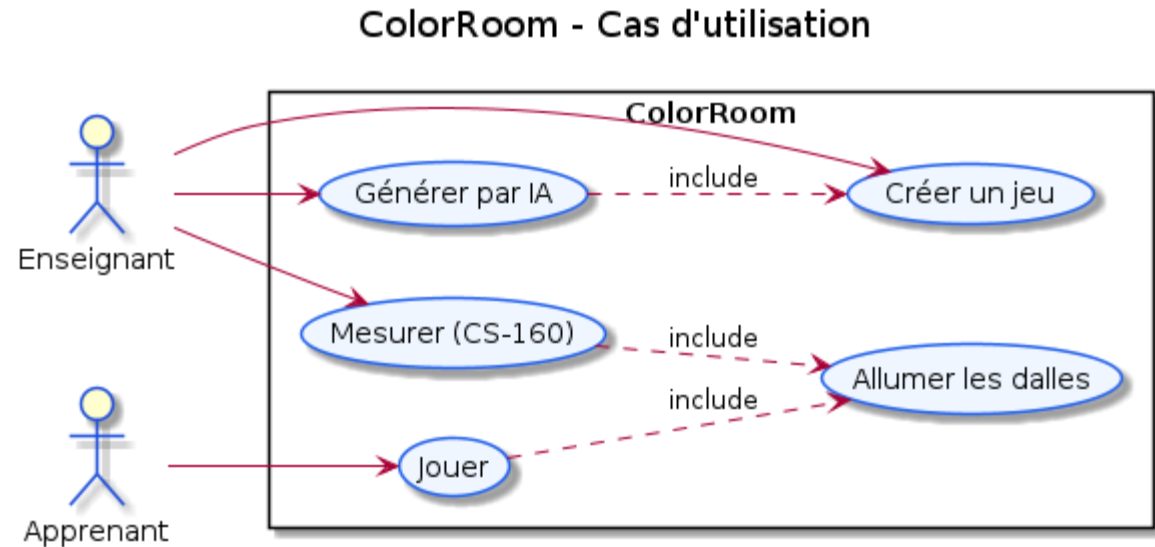
## Contraintes

- **Raspberry Pi 5** + SSD, Docker imposé
- Fonctionnement **hors-ligne** (réseau local)
- Latence des plaques à compenser



# Diagramme de cas d'utilisation

- Deux acteurs : **Enseignant** (crée et génère des jeux) et **Apprenant** (joue)
- Les deux peuvent **Mesurer** avec le colorimètre CS-160
- *Créer un jeu* **inclut** *Générer par IA*
- *Jouer* et *Mesurer* **incluent** *Allumer les dalles*





# Équipe et répartition

## E1 · Maxime Bonnevey

Infrastructure Pi & Docker, CI/CD, intégration colorimètre, sécurité

## E2 · Téo Trompier

MOI

Base de données, API, UI/UX, jeux solo + multijoueur, proxy LED, documentation

## E3 · Ilyes Arbadji

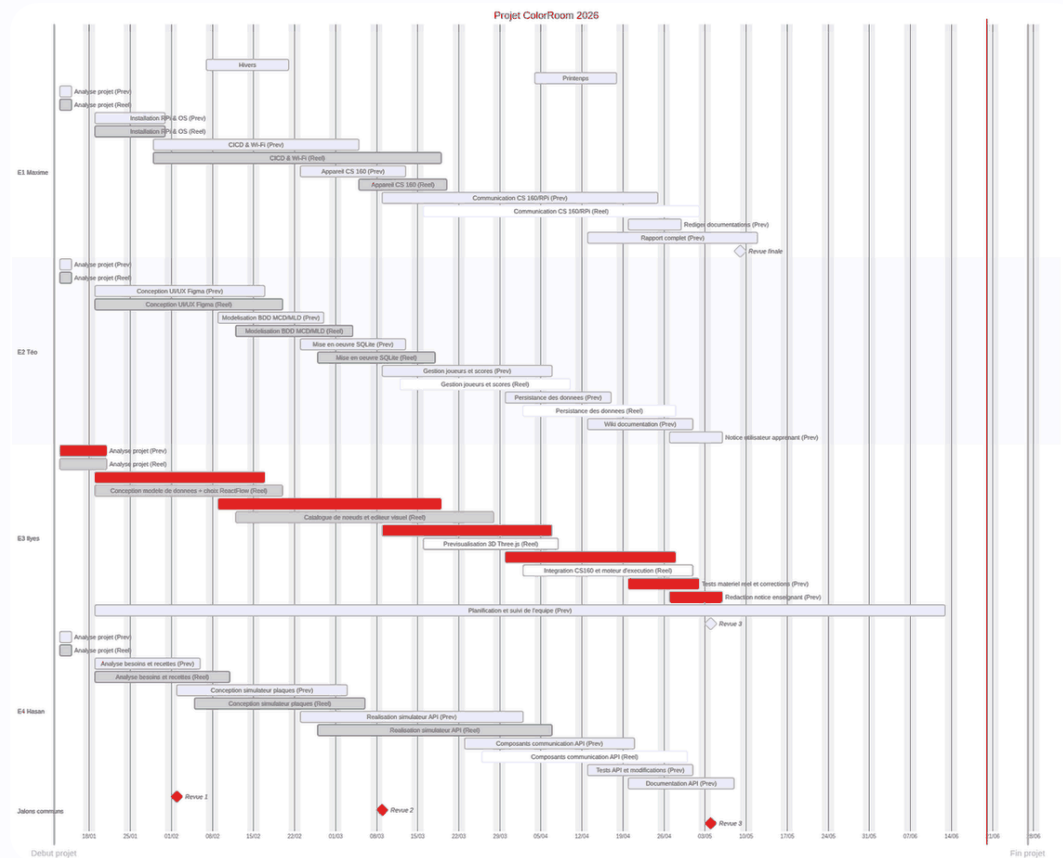
Éditeur de jeux (hors de mon périmètre)

## E4 · Hasan Akyuz

Tests de l'API, simulateur de plaques, Swagger / Postman



# Diagramme de Gantt



Jalons par étudiant · ma contribution (E2) suivie tout au long du projet

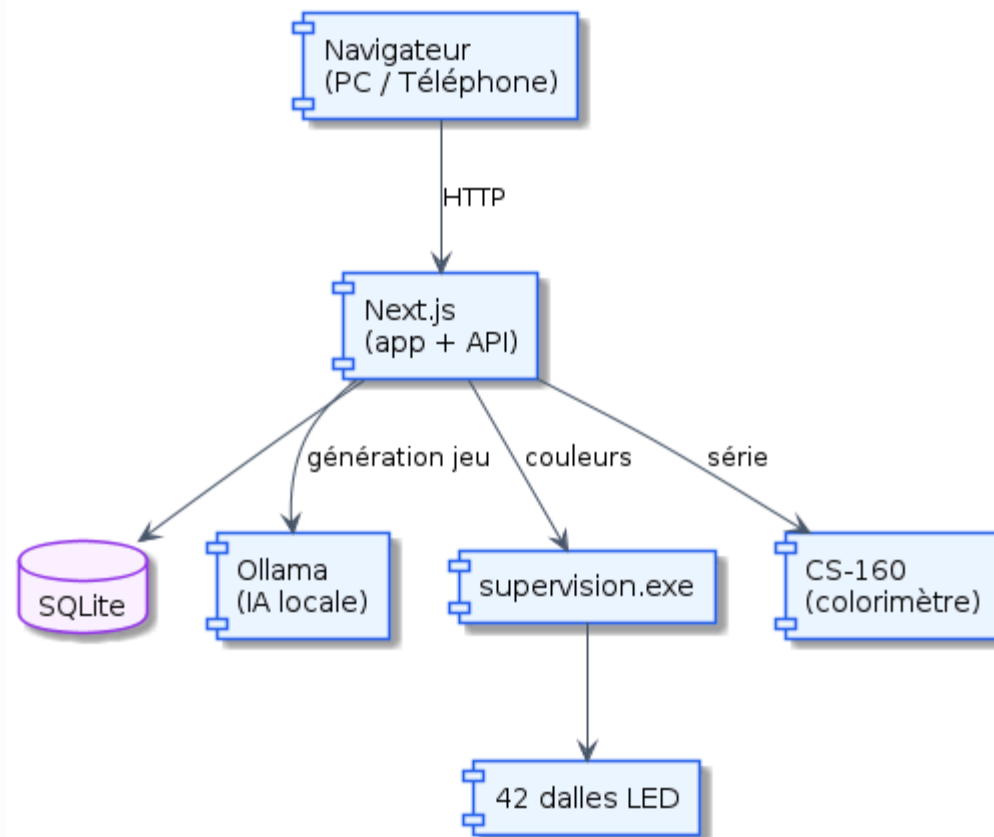




# Architecture logicielle

- Serveur unique **Next.js** (App Router) conteneurisé
- **Route Handlers** + couche **lib/** (auth, db, services)
- Proxy HTTP de **supervision** ; pont **.NET** du CS-160
  - SQLite (better-sqlite3) · 100 % hors-ligne

## ColorRoom - Architecture

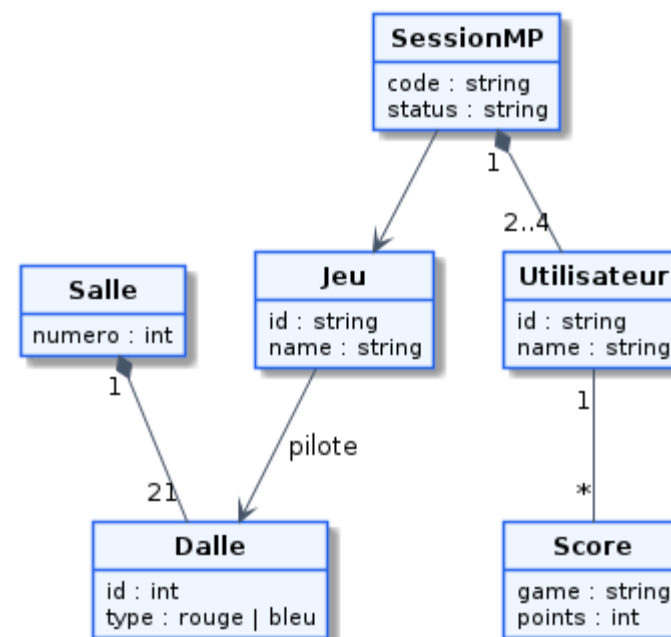




# Diagramme de classes

- Entités métier modélisées et **typées en TypeScript**
- Utilisateur, Classe, Jeu, Score, Session...
- Relations **1-N** (une classe regroupe plusieurs apprenants)
- Vue **objet** du domaine, complémentaire du modèle relationnel

ColorRoom - Classes principales





# Choix techniques

---

## Piste initiale : Node-RED

- Modèle **flow-based** : mal adapté à une appli multi-pages
- Pas de composants visuels pour une UI/UX riche
- Flux illisibles à grande échelle

## Choix retenu : Next.js + React + TS

- **React** : composants réutilisables, DOM virtuel, état réactif
- **TypeScript strict** : erreurs à la compilation (tsc + ESLint)
- **Next.js** : Route Handlers natifs, un seul runtime Node



# La pile technique



**Next.js**

16.2 · App Router



**React**

19 · composants



**TypeScript**

5.5 · strict



**Node.js**

runtime serveur



**SQLite**

better-sqlite3 11.5



**Three.js**

0.160 · vue 3D



**Docker**

multi-stage arm64



**Raspberry Pi**

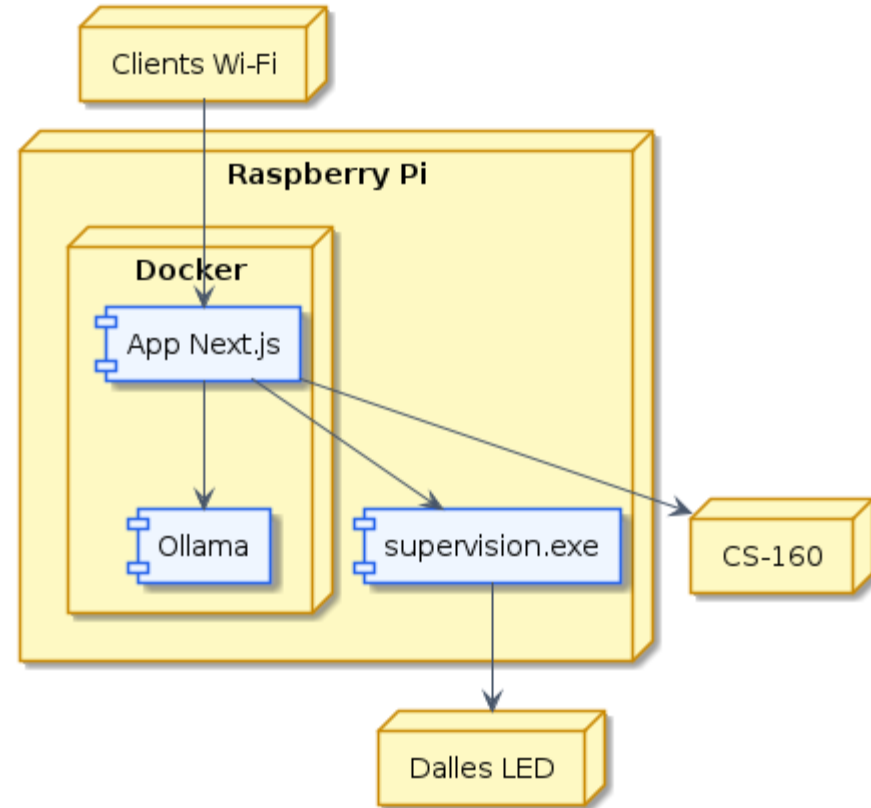
5 · cible



# Réseau et déploiement

- Image **Docker multi-stage** (deps → builder → runner), **arm64**
- App supervisée par **Portainer** · dalles pilotées en **RS-485**
- Wi-Fi local **ColorRoom\_WiFi** ; **CS-160** en USB
- Accès depuis tout appareil : **<http://172.17.40.39/>**
  - SQLite en volume · git pull puis docker compose up

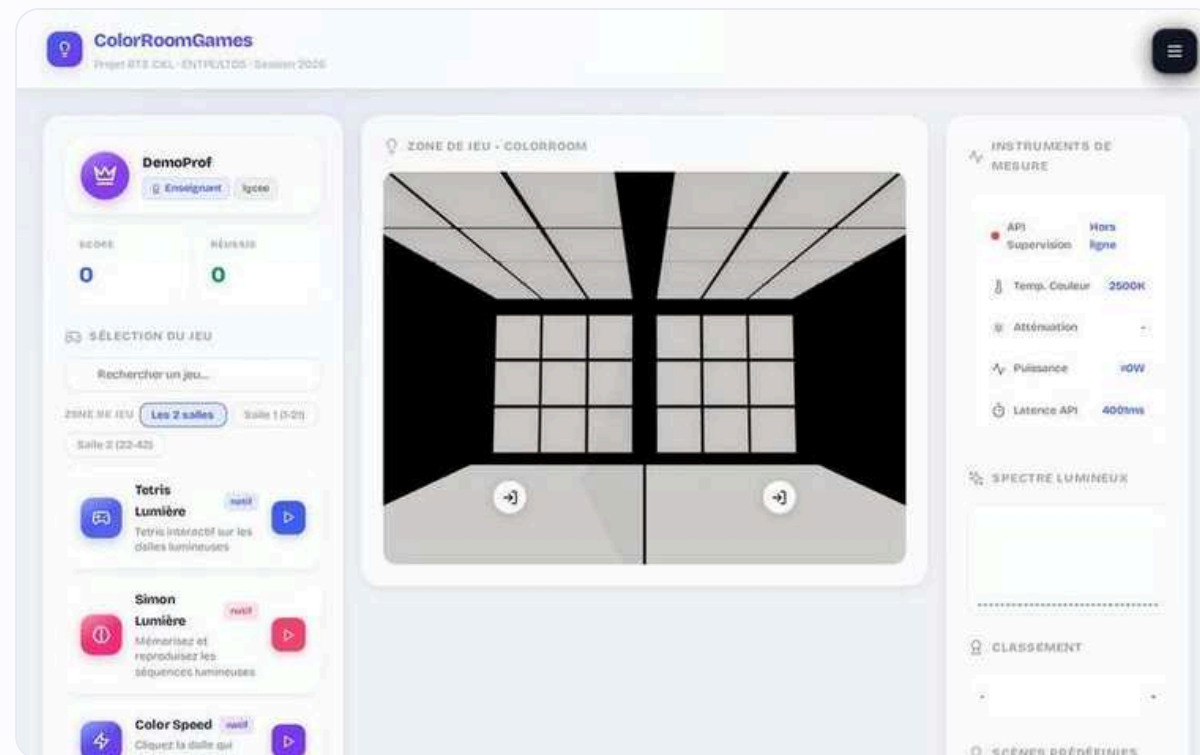
## ColorRoom - Déploiement





# Interface et design system

- Design system « **verre dépoli** » : variables CSS partagées
- Pages : accueil 3D, /jeux, /mesure, /chromaticite, /gestion, /aide
- **Menu dynamique** selon le rôle
- **Vue 3D** (Three.js : WebGLRenderer, Room3D, 2 cellules)
  - forceContextLoss au démontage · pause via Page Visibility API

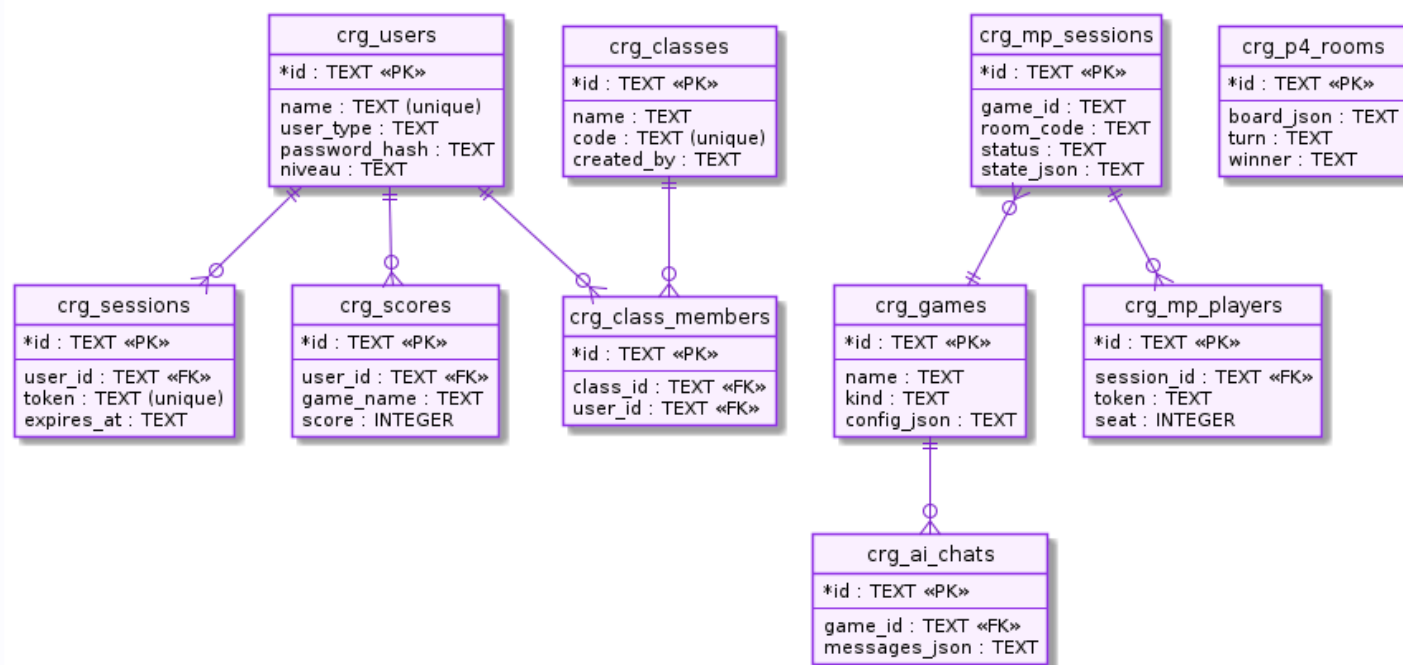




# Base de données SQLite

- **better-sqlite3** (API synchrone, requêtes préparées)
- Clés étrangères **ON DELETE CASCADE**
- Migrations **idempotentes** au démarrage
- **journal\_mode=WAL** + busy\_timeout

ColorRoom - Schéma de base de données (SQLite)





# Transaction atomique (ACID)

- L'inscription crée l'utilisateur **et** son adhésion de classe
- **Tout-ou-rien** : si une étape échoue, rien n'est écrit (pas de compte « à moitié créé »)
- Garanti par `db.transaction()` de better-sqlite3

[app/api/auth/register/route.ts](#)

`</>` [github.com/NewGenesisAgency/color\\_room/blob/main/app/api/auth/register/route.ts](https://github.com/NewGenesisAgency/color_room/blob/main/app/api/auth/register/route.ts)  
L58

```
app/api/auth/register/route.ts

// Tout-ou-rien : si la jonction de classe échoue,
// l'utilisateur n'est pas créé non plus.
const insertAll = db.transaction() => {
  db.prepare("INSERT INTO crg_users ...").run(...);
  if (classCode) db.prepare("INSERT OR IGNORE
    INTO crg_class_members ...").run(...);
};
insertAll(); // exécution atomique (ACID)
```





# Gestion de l'état



## Persistante

Stockée durablement en SQLite (survit aux redémarrages, volume Docker). `lib/db`



## Transactionnelle · atomique

`db.transaction()` : ACID, tout-ou-rien (user + classe).  
`register`



## Volatile (en mémoire)

`useRef` : état runtime des dalles/jeux, perdu au rechargement. `appl/jeux`



## Environnement

`process.env` : secrets (clé, mot de passe admin) via `.env`, hors Git.



## Réactive

`useState` : déclenche le re-rendu de l'interface à chaque changement.



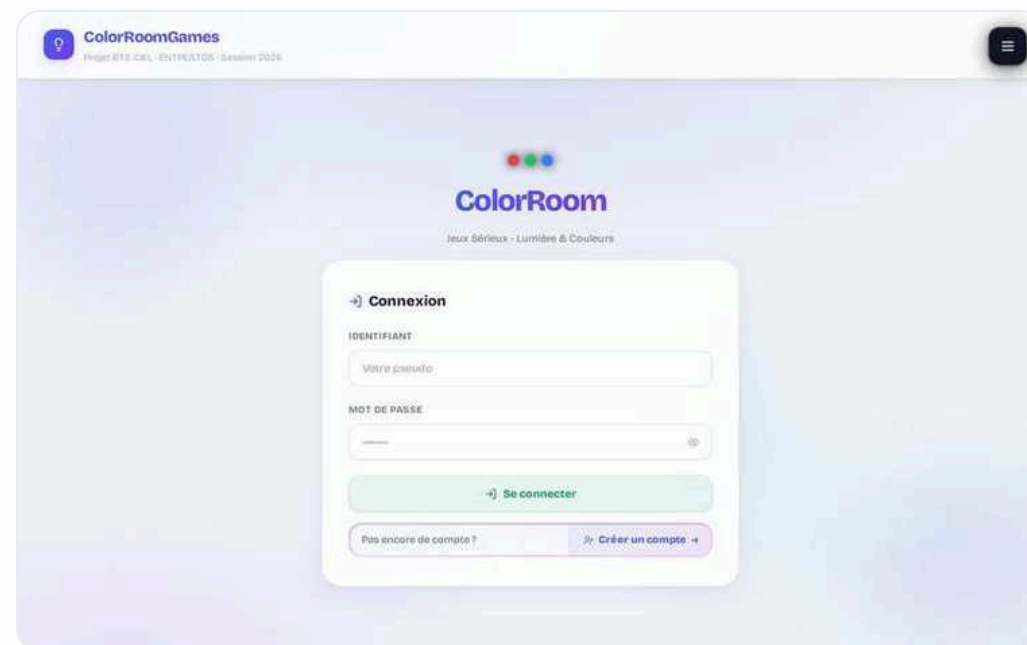
## Concurrente

Sémaphore `HW_CONCURRENCY=2` : sérialise les accès au matériel. `batch`



# Authentification et données personnelles

- **3 rôles** : admin (via .env), enseignant, apprenant
- Hachage **PBKDF2-HMAC-SHA512** (100 000 itér., sel 16 o, clé 64 o)
- Session en **cookie HttpOnly + SameSite=lax** (anti-XSS / CSRF)
- **Données 100 % locales** ; minimisation, effacement en cascade





# Hachage des mots de passe (PBKDF2)

- Dérivation lente **PBKDF2-HMAC-SHA512**, 100 000 itérations
- **Sel aléatoire** de 16 octets par compte (anti rainbow-tables)
- Stockage au format **sel:hash** ; le clair n'est jamais conservé

[app/lib/auth.ts](#)

</> [github.com/NewGenesisAgency/color\\_room/blob/main/app/lib/auth.ts#L19-L23](https://github.com/NewGenesisAgency/color_room/blob/main/app/lib/auth.ts#L19-L23)

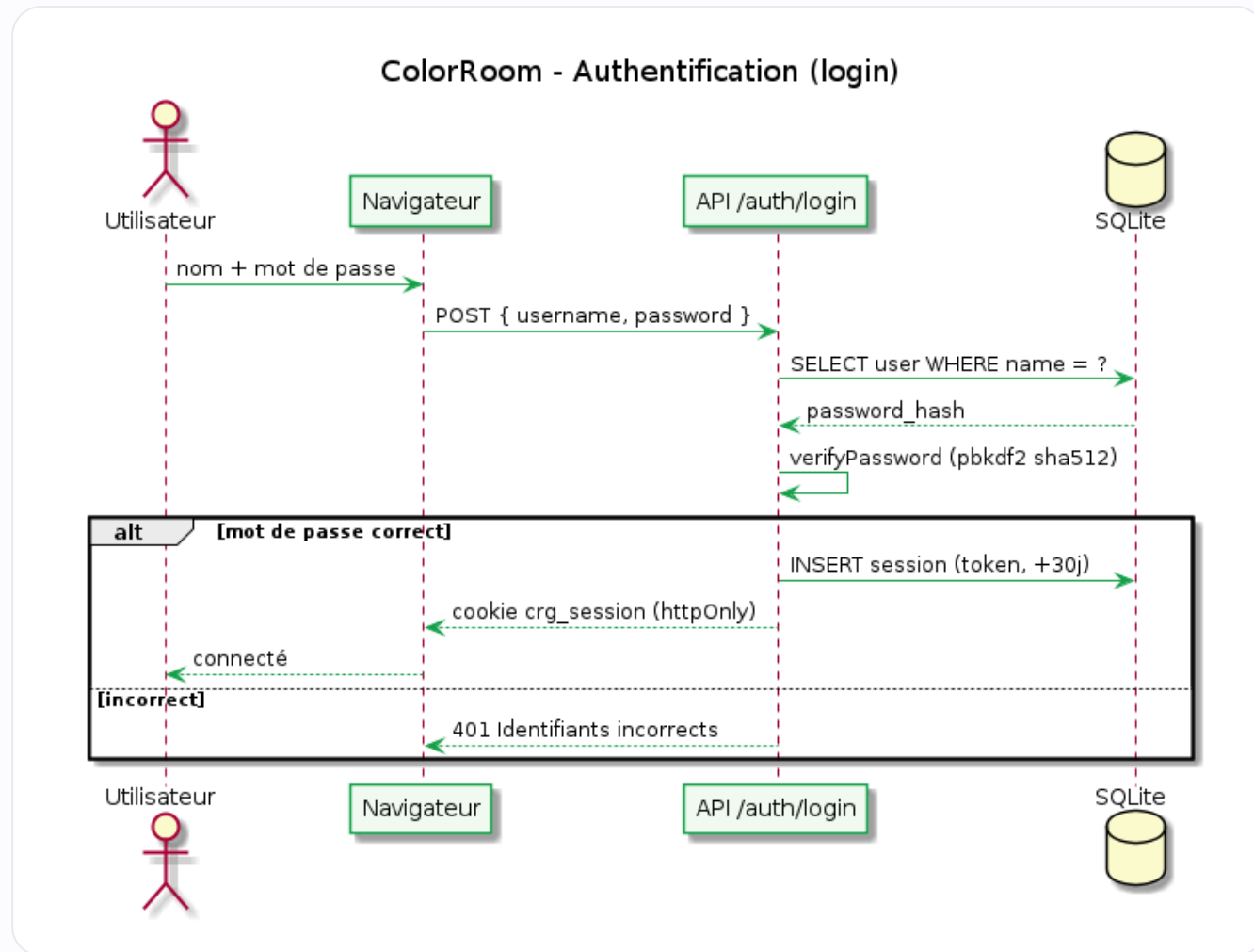
app/lib/auth.ts

```
export function hashPassword(password: string) {  
  const salt = randomBytes(16).toString('hex');  
  const hash = pbkdf2Sync(password, salt,  
    100_000, 64, 'sha512').toString('hex');  
  return `${salt}:${hash}`; // format sel:hash  
}
```



# Authentication (PBKDF2 + cookie)

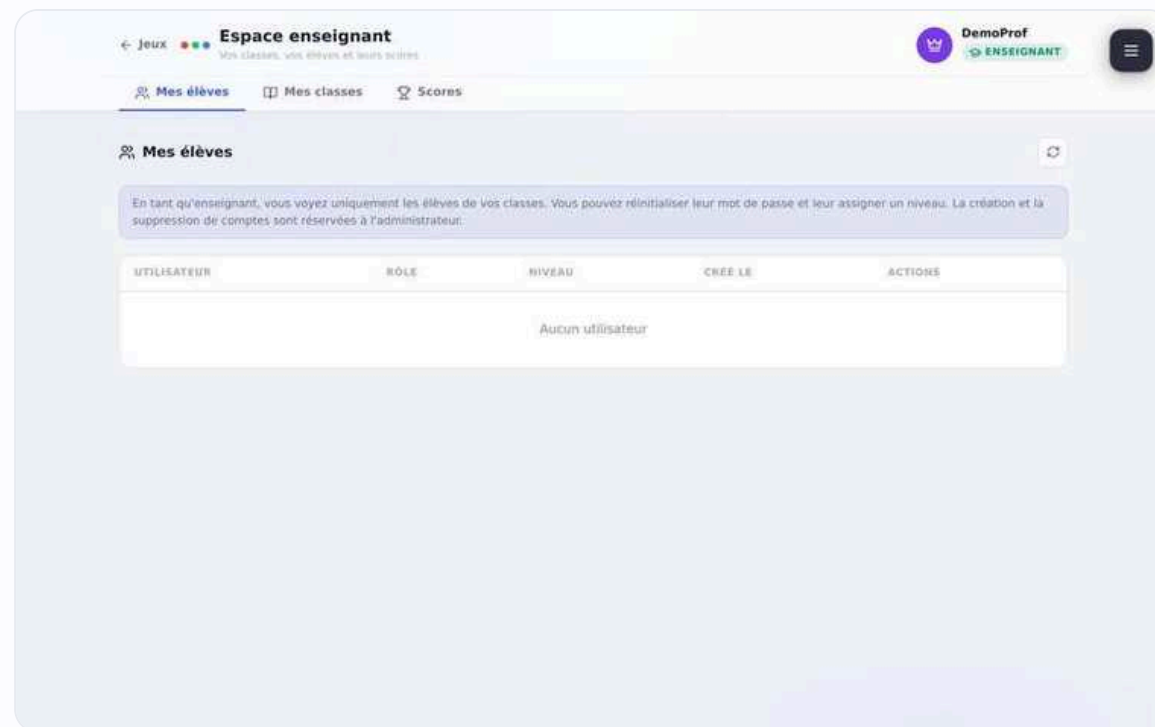
- L'apprenant saisit identifiant + mot de passe
- L'API recalcule le hash via **verifyPassword** (PBKDF2)
- Si valide, création d'une **session** (token aléatoire)
- Renvoi d'un cookie **HttpOnly + SameSite** (30 j glissants)





# Comptes, classes, scores et tableau de bord

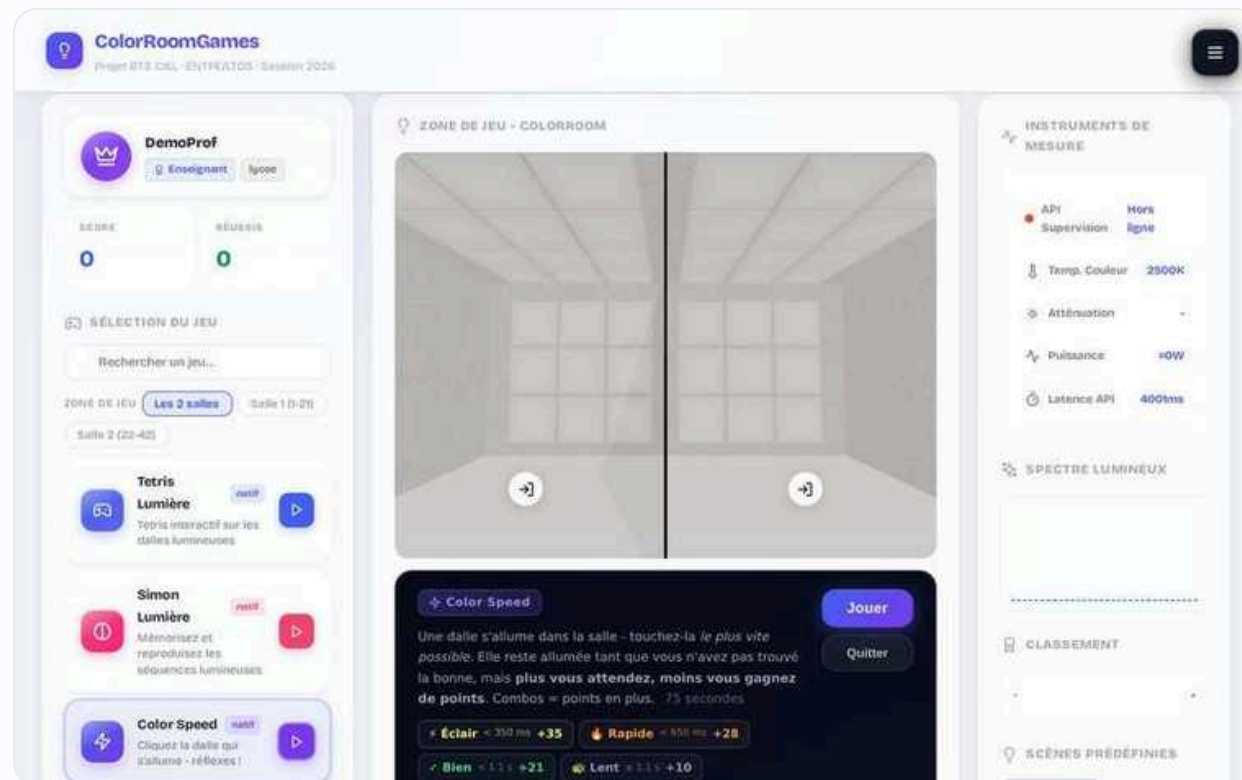
- **Classes** : code 6 caractères (sans 0/O, 1/I)
- **Niveaux** assignés par l'enseignant
- **Scores** : JOIN scores × users ; **all=1** réservé (HTTP 403)
  - Tableau de bord /gestion · export CSV (Blob, BOM UTF-8)





# Les jeux et le pilotage des dalles

- Tetris, Simon, Maître du Blanc, Color Speed...
- Pilotage des **32 canaux/dalle** en parallèle (Promise.all)
- Timeout via **AbortController** ; couleur écran = couleur dalle
  - CHANNEL\_PROFILES calés sur la longueur d'onde réelle

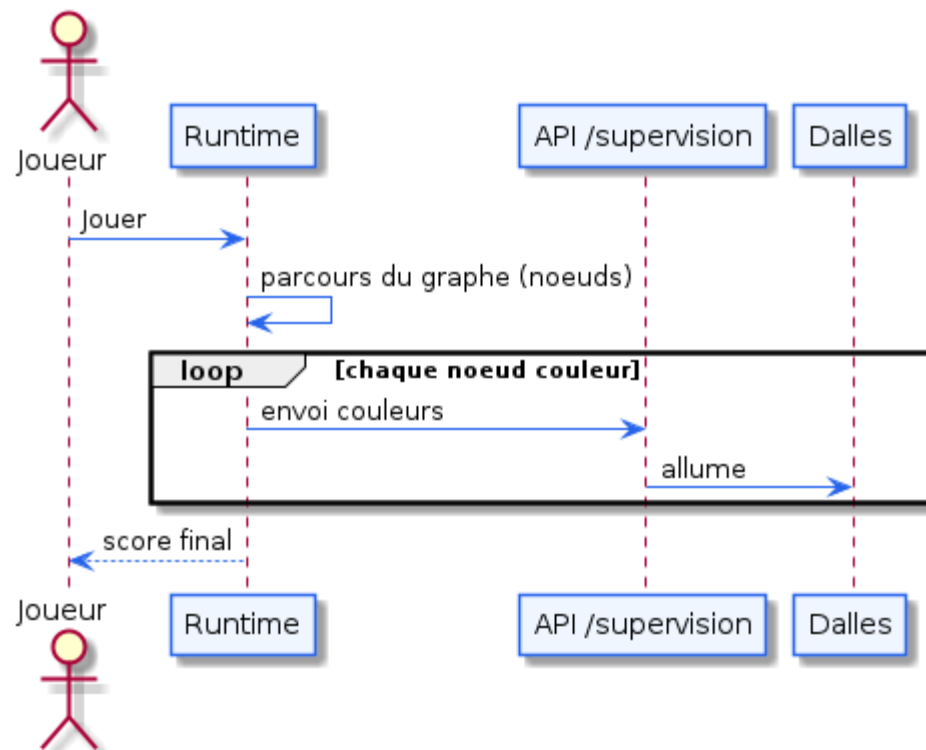




# Du clic joueur aux dalles

- Le joueur lance une partie depuis le catalogue
- Le **runtime** parcourt le graphe de nœuds du jeu
- Chaque nœud couleur appelle **/api/supervision**
- Les **dalles** s'allument ; le score remonte à l'écran

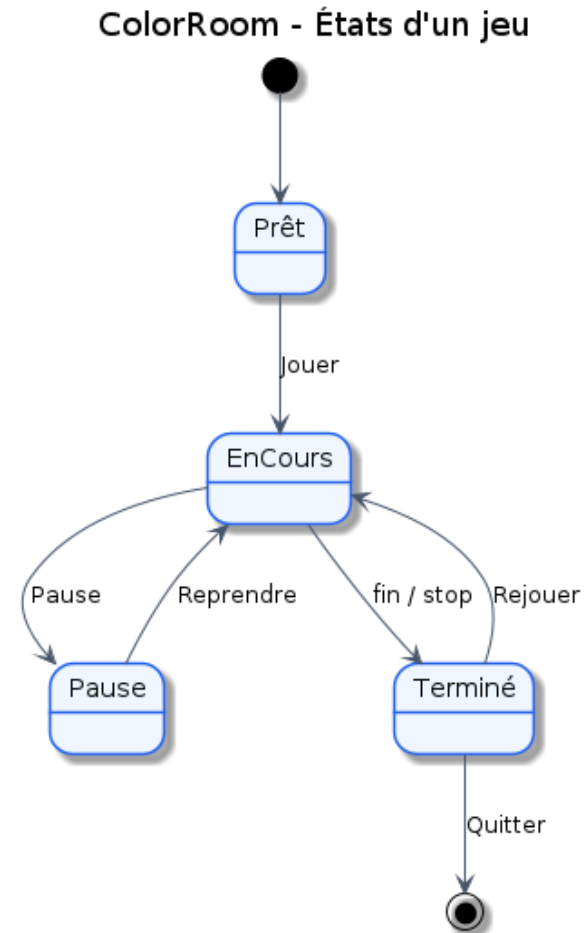
## ColorRoom - Exécution d'un jeu





# Cycle de vie d'une partie

- États : **Prête** → **En cours** → **Terminée**
- Transitions : démarrer, jouer un coup, fin de partie
- Calcul et enregistrement du **score** en fin de partie
- Réinitialisation pour **rejouer**

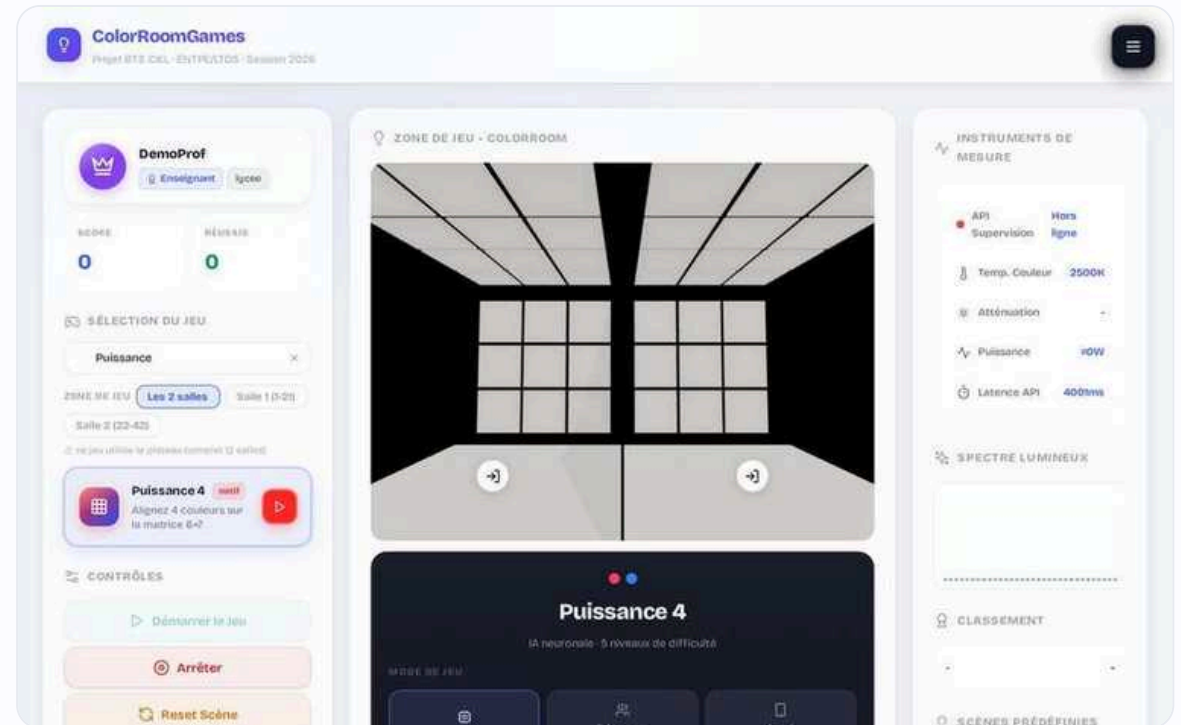






# Puissance 4 et son IA minimax

- Grille 6 colonnes × 7 lignes (42 cases) ; 2 joueurs ou contre l'ordinateur
- IA **hors-ligne** : **minimax** + **élagage alpha-bêta**, anti-piège
- Heuristique par **fenêtres de 4** (défense pondérée > attaque) + poids central
- **5 niveaux** : profondeur **1 / 2 / 5 / 9 / 12** + bruit décroissant





# Évaluation minimax (alpha-bêta)

- Chaque fenêtre de 4 cases est notée du point de vue de l'IA
- **Défense > attaque** : un alignement adverse de 3 vaut -170, le mien +130
- Victoire = **WIN\_SCORE** (1 000 000) ; recherche bornée en profondeur

[app/\\_components/GamePuissance4.tsx](#)

</> [github.com/NewGenesisAgency/color\\_room/blob/main/app/\\_components/GamePu](#)  
L128

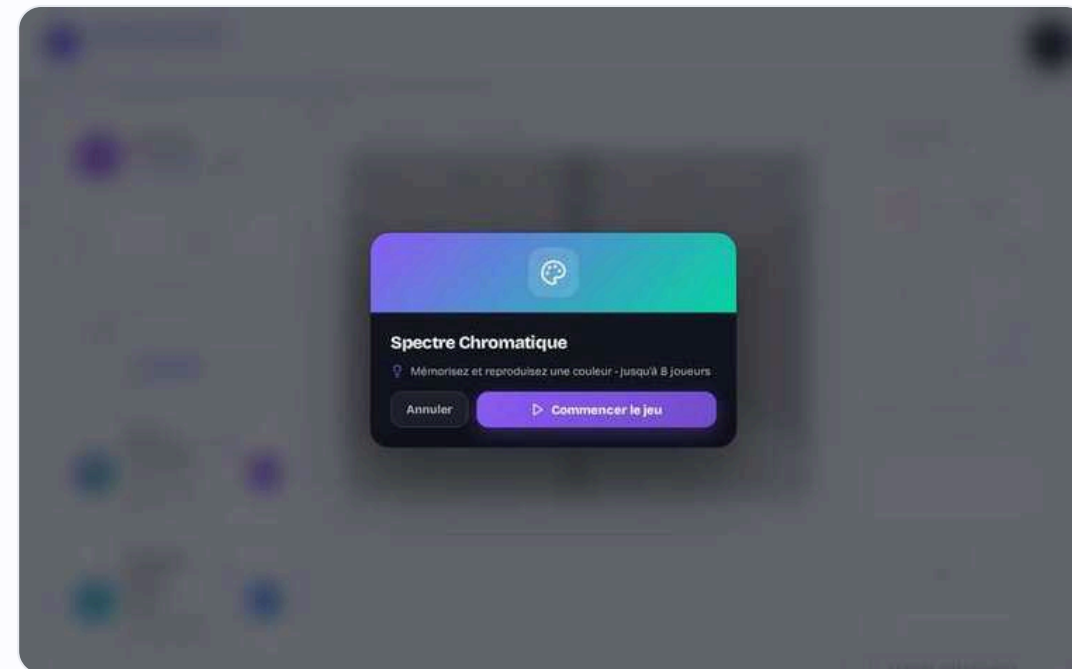
```
app/_components/GamePuissance4.tsx

// Note d'une fenêtre de 4 (défense > attaque)
function scoreWindow(me, opp) {
  if (me>0 && opp>0) return 0; // fenêtre morte
  if (me===4) return WIN_SCORE; // 1 000 000
  if (me===3) return 130;
  if (opp===3) return -170; // bloque la menace
  ...
}
// minimax + alpha-bêta · profondeur 1 → 12
```



# Les jeux multijoueur

- État persisté **en base**, récupéré par **polling** de `/state`
- **Spectre Chromatique** : jusqu'à 8 joueurs (score = distance)
- **Morpion en réseau** : 2 joueurs, détection de victoire
- **Heartbeat** de présence · jetons **UUID** · hôte = siège 1

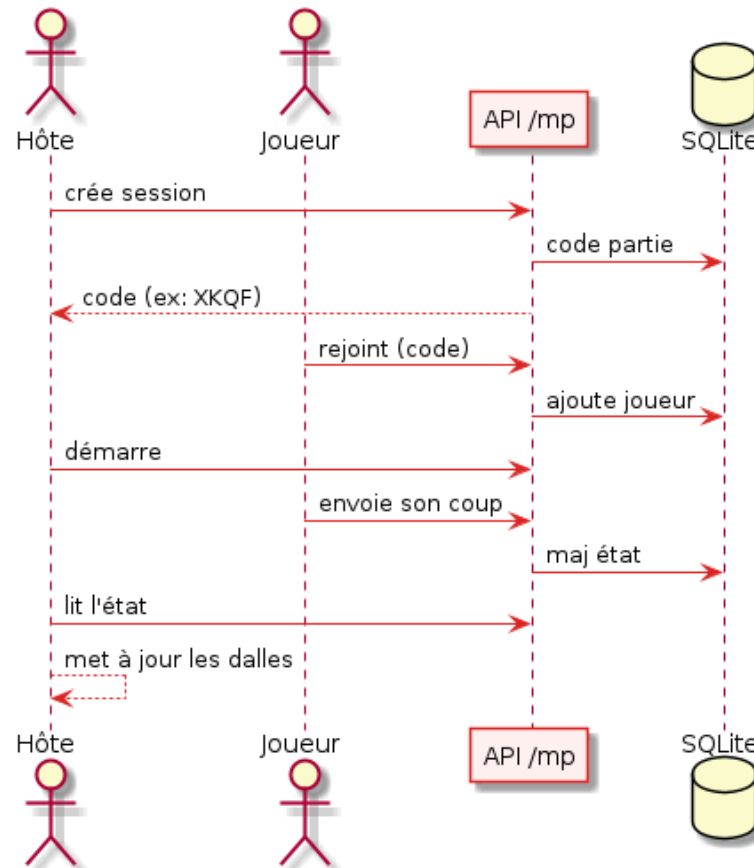




# État partagé et interrogation périodique

- L'hôte crée une session et obtient un **code**
- Les invités rejoignent en saisissant ce code
- L'état est **persisté en base** (state\_json)
- Chaque client interroge **/state** en **polling** → écrans synchronisés

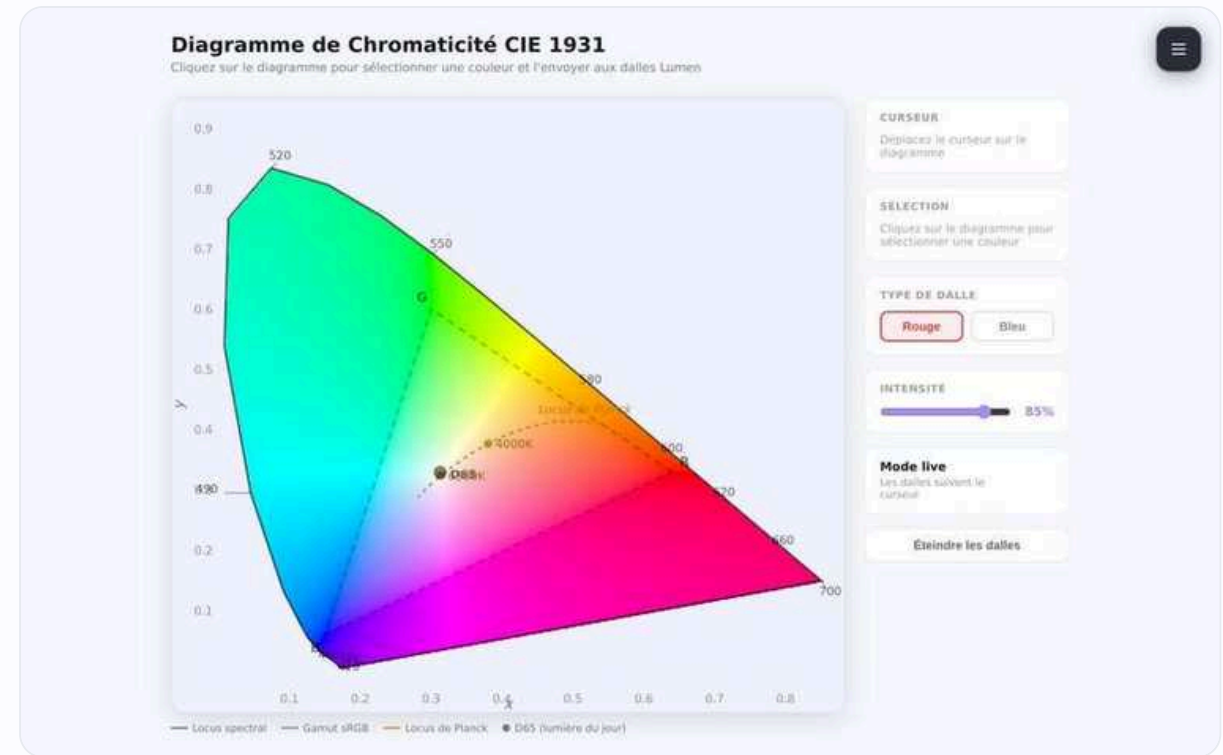
ColorRoom - Session multijoueur





# Mesure colorimétrique et chromaticité

- Colorimètre **Konica Minolta CS-150/160** via pont .NET ( [/api/CS160](#) )
- Tristimulus **CIE XYZ**, luminance **L<sub>v</sub> (cd/m<sup>2</sup>)**, chromaticité (**x**, **y**)
- Tracé sur le **diagramme CIE 1931** ; écart **ΔE** pour le score
- 32 canaux = **24 spectres étroits** (proche UV → proche IR) + **8 blancs**

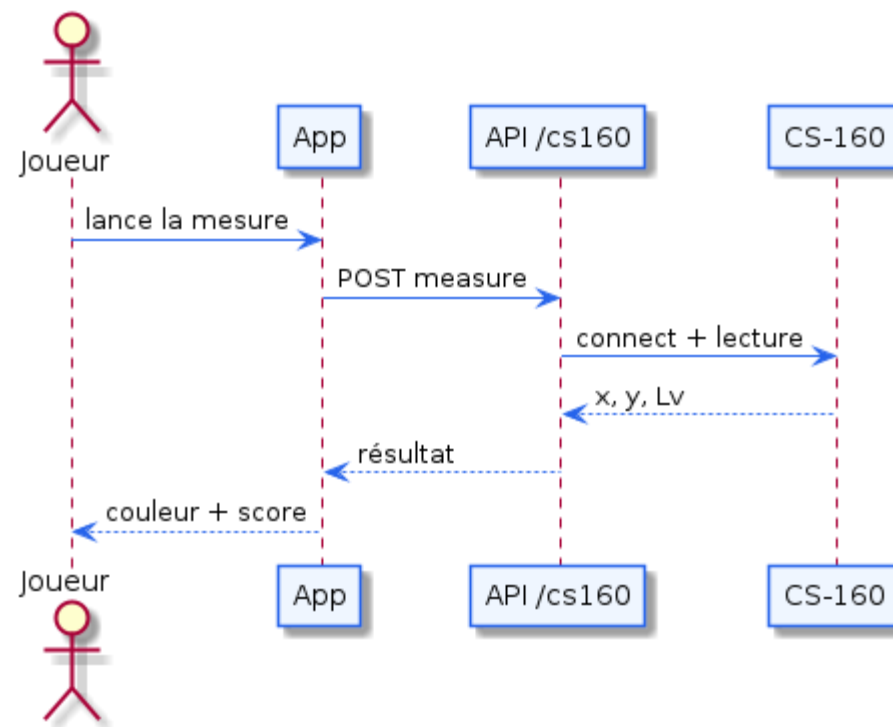




# Pilotage du colorimètre CS-160

- Connexion au CS-160 via le **pont .NET** (/api/CS160)
- Allumage de la **dalle cible** à mesurer
- Mesure : **tristimulus XYZ**, **Lv**, chromaticité (**x, y**)
- Tracé du point sur le **diagramme CIE 1931**

## ColorRoom - Mesure CS-160

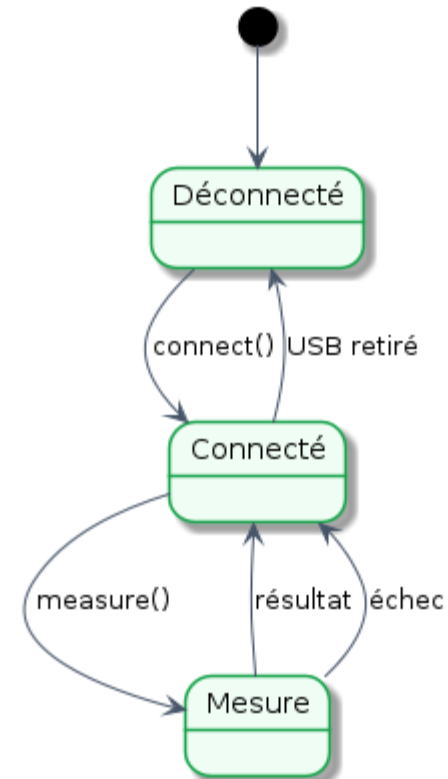




# Cycle de mesure du CS-160

- États : **Déconnecté** → **Connecté**
- **Mesure en cours** → **Résultat** disponible
- Gestion des **erreurs** (timeout, appareil absent)
- Retour à l'état prêt pour une nouvelle mesure

ColorRoom - États du colorimètre CS-160

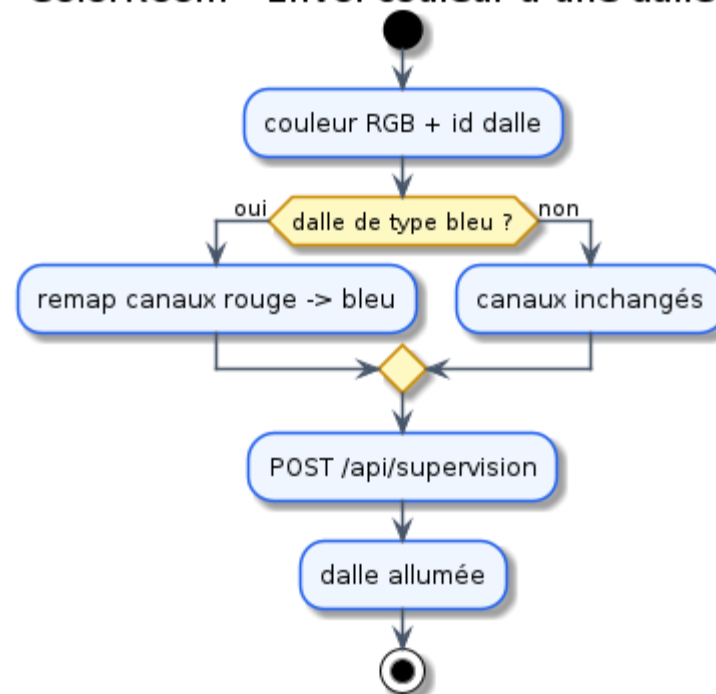




# Remappage des canaux LED

- Entrée : une couleur **RGB** demandée par le jeu
- Conversion vers les **32 canaux** de la plaque
- Application des **profils** (longueur d'onde réelle)
- Envoi de la trame au matériel (proxy supervision)

## ColorRoom - Envoi couleur à une dalle

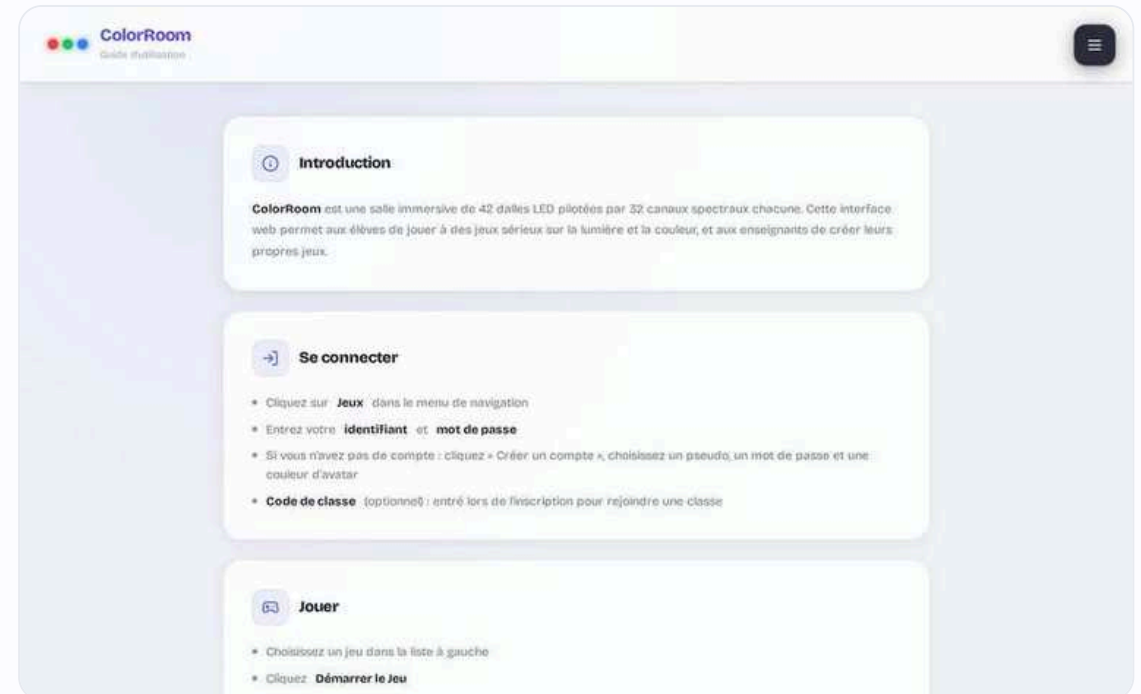






# Documentation et robustesse

- Notice apprenant intégrée (page /aide)
- Manuel d'installation (README) : Docker, dev, service permanent
- Migrations idempotentes · **transactions ACID**
  - Connexion SQLite en singleton · exports UTF-8 + BOM





# Difficultés rencontrées et solutions

Latence des plaques (32 canaux)



✓ Envoi parallèle (Promise.all) + timeout (AbortController)

Couleur écran différente des dalles



✓ Rendu unifié + profils calés sur les longueurs d'onde

Accès concurrents (SQLITE\_BUSY)



✓ Mode WAL + busy\_timeout + connexion singleton

Multijoueur temps réel sur Pi



✓ État en base + polling /state (vs WebSockets)



# Bilan et perspectives

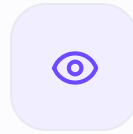
---

## Objectifs E2 atteints

- Interface, base de données et comptes opérationnels
- Jeux solo, IA Puissance 4 et multijoueur
- Mesure CS-160 et diagramme CIE
- Documentation et déploiement Docker

## Perspectives

- Salons multijoueur avec codes de partie
- Nouveaux jeux pédagogiques et tutoriels
- Tests automatisés (CI/CD)
  - Exploration : génération de jeux par IA locale (Ollama)



PLACE À LA

# Démonstration

Connexion · catalogue et un jeu sur les dalles · Puissance 4 contre l'IA · un jeu multijoueur · mesure CS-160 et diagramme CIE · tableau de bord enseignant



# Merci de votre attention

Avez-vous des questions ?